Comparative Analysis of Optimisation Algorithms in Physics-Informed Neural Networks

A Numerical Study of the Two-Dimensional Laplace Equation

Subham

November 2026

Abstract

This project investigates the use of Physics-Informed Neural Networks (PINNs) for approximating the solution of a two-dimensional Laplace boundary-value problem. A fully connected neural network is implemented in PyTorch, while automatic differentiation is used to evaluate the second-order spatial derivatives appearing in the governing partial differential equation. The main objective is to compare the optimisation behaviour and numerical accuracy obtained using Adam and Limited-memory Broyden–Fletcher–Goldfarb–Shanno (L-BFGS).

The computational domain is the unit square, with homogeneous Dirichlet conditions on three boundaries and a sinusoidal Dirichlet condition on the top boundary. Since an analytical solution is available, the network predictions can be evaluated using mean absolute error, maximum absolute error, relative L^2 error, and spatial error plots.

The experiments use 8,000 interior collocation points and 2,000 boundary points, giving 10,000 training points in total. Both optimisation runs use 10,000 training epochs or outer iterations, respectively. In the reported runs, Adam uses a learning rate of 0.001, while L-BFGS uses a step-size parameter of 1.0 together with a strong-Wolfe line search. The measured results show that L-BFGS achieves lower mean absolute and relative L^2 errors in these particular runs, whereas Adam gives a slightly lower maximum pointwise error. The study therefore compares not only loss convergence, but also the accuracy of the resulting approximate solutions.

Contents

1 Introduction and Aim of the Project **5**

1.1 Background 5

1.2 Main Aim 5

2 Mathematical Formulation **7**

2.1 Governing Equation 7

2.2 Boundary Conditions 7

2.3 Analytical Solution 8

2.4 Uniqueness and Well-Posedness 8

3 Physics-Informed Neural Network **9**

3.1 Network Architecture 9

3.2 Automatic Differentiation 9

3.3 Loss Function 10

4 Optimisation Algorithms Studied **11**

4.1 Gradient-Based Optimisation 11

4.2 Adam Optimiser 11

4.3 L-BFGS Optimiser 12

4.4 Why Compare Adam and L-BFGS? 12

5 Dataset and Experimental Configuration **13**

5.1 Training Dataset 13

5.2 Experimental Hyperparameters 14

5.3 Boundary Sampling Note 14

6 Results and Analysis **15**

6.1 L-BFGS: Predicted Solution and Error 15

6.2 Adam: Predicted Solution and Error 15

6.3 L-BFGS Training Convergence 16

6.4 Adam Training Convergence 17

6.5 Quantitative Validation 17

6.6 Interpretation of Validation Results 18

7 Discussion **19**

7.1 Observed Optimisation Behaviour 19

7.2 Limitations 19

7.3 Further Work 20

8 Conclusion **21**

List of Figures

6.1 Analytical solution, PINN prediction and absolute error for the L-BFGS run with step-size parameter 1.0. 15
6.2 Analytical solution, PINN prediction and absolute error for the Adam run with learning rate 0.001. 16
6.3 L-BFGS convergence history over 10,000 outer iterations. 16
6.4 Adam convergence history over 10,000 epochs. 17

List of Tables

5.1 Training dataset composition 13
5.2 Optimisation configurations used in the reported runs 14

6.1 Numerical validation results 18

Chapter 1

Introduction and Aim of the Project

1.1 Background

Partial differential equations (PDEs) describe a wide range of physical phenomena, including heat conduction, electrostatics, potential flow and many problems in continuum mechanics. Traditional numerical approaches include finite-difference, finite-element and finite-volume methods.

Physics-Informed Neural Networks provide an alternative computational framework in which the governing differential equation and its boundary conditions are incorporated into a neural-network training objective. Rather than relying exclusively on labelled input-output data, a PINN is trained to reduce the residual of the governing equation while satisfying the prescribed boundary conditions.

1.2 Main Aim

The main aim of this project is to implement a PINN for the two-dimensional Laplace equation and examine how different optimisation algorithms affect its training behaviour and numerical accuracy.

The specific objectives are:

1. Formulate a two-dimensional Laplace boundary-value problem with a known analytical solution.
2. Implement the governing differential operator using PyTorch automatic differentiation.
3. Construct a physics-informed loss function combining the PDE residual and boundary-condition mismatch.
4. Study the optimisation principles behind Adam and L-BFGS before implementing them.

5. Compare the convergence behaviour of the two algorithms.

6. Evaluate the predicted solutions against the analytical solution using quantitative error metrics and contour plots.

The project is intended as a focused computational and numerical study. It does not claim to introduce a new PINN architecture or a new optimisation algorithm.

Chapter 2

Mathematical Formulation

2.1 Governing Equation

The two-dimensional Laplace equation is

$$\boxed{\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0} \tag{2.1}$$

over the domain

$$\Omega = [0,1] \times [0,1]. \tag{2.2}$$

The operator

$$\Delta u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \tag{2.3}$$

is called the Laplacian.

2.2 Boundary Conditions

The Dirichlet boundary conditions used are

$$u(0, y) = 0, \tag{2.4}$$
$$u(1, y) = 0, \tag{2.5}$$
$$u(x, 0) = 0, \tag{2.6}$$
$$u(x, 1) = \sin(\pi x). \tag{2.7}$$

These conditions prescribe the value of the solution on all four boundaries of the unit square.

2.3 Analytical Solution

For this problem, an analytical solution is

$$\boxed{u(x,y) = \frac{\sinh(\pi y)}{\sinh(\pi)} \sin(\pi x)} \tag{2.8}$$

This expression satisfies the four boundary conditions. It also satisfies the Laplace equation because

$$u_{xx} = -\pi^2 \frac{\sinh(\pi y)}{\sinh(\pi)} \sin(\pi x), \tag{2.9}$$

$$u_{yy} = \pi^2 \frac{\sinh(\pi y)}{\sinh(\pi)} \sin(\pi x). \tag{2.10}$$

Therefore,

$$u_{xx} + u_{yy} = 0. \tag{2.11}$$

The analytical solution is important because it provides an independent reference for evaluating the PINN prediction.

2.4 Uniqueness and Well-Posedness

The Dirichlet problem for Laplace's equation has a unique solution under the usual regularity assumptions. Consequently, if a sufficiently accurate approximation satisfies both the governing equation and the prescribed boundary conditions, it is approximating a well-defined target solution.

This mathematical property motivates the use of the PDE residual and boundary mismatch as components of the training objective.

Chapter 3

Physics-Informed Neural Network

3.1 Network Architecture

A fully connected feedforward neural network is used:

$$2 \longrightarrow 32 \longrightarrow 32 \longrightarrow 32 \longrightarrow 1. \tag{3.1}$$

The two inputs are the spatial coordinates (x, y), and the scalar output is the approximate solution $u_\theta(x, y)$. The hidden layers use the hyperbolic tangent activation function, while the output layer is linear.

3.2 Automatic Differentiation

The network prediction is denoted by $u_\theta(x, y)$. PyTorch automatic differentiation is used to calculate

$$u_x = \frac{\partial u_\theta}{\partial x}, \qquad u_y = \frac{\partial u_\theta}{\partial y}, \tag{3.2}$$

$$u_{xx} = \frac{\partial^2 u_\theta}{\partial x^2}, \qquad u_{yy} = \frac{\partial^2 u_\theta}{\partial y^2}. \tag{3.3}$$

The pointwise PDE residual is

$$r_\theta(x, y) = u_{xx}(x, y) + u_{yy}(x, y). \tag{3.4}$$

The implementation uses differentiable coordinate tensors and higher-order automatic differentiation so that the PDE residual can also be differentiated with respect to the trainable network parameters.

3.3 Loss Function

The PDE loss is defined as

$$L_{\text{PDE}} = \frac{1}{N_i} \sum_{j=1}^{N_i} \left[r_\theta(x_j, y_j) \right]^2 , \tag{3.5}$$

where (x_j, y_j) are interior collocation points.

The boundary loss is

$$L_{\text{BC}} = \frac{1}{N_b} \sum_{j=1}^{N_b} \left[u_\theta(X_j) - U_j \right]^2 , \tag{3.6}$$

where X_j denotes a boundary coordinate and U_j its prescribed boundary value.

The total loss is

$$\boxed{L_{\text{total}} = L_{\text{PDE}} + \lambda_{\text{BC}} L_{\text{BC}}} \tag{3.7}$$

with

$$\lambda_{\text{BC}} = 10. \tag{3.8}$$

Chapter 4

Optimisation Algorithms Studied

4.1 Gradient-Based Optimisation

Training a neural network requires finding parameters θ that minimise the loss function:

$$\theta^* = \arg\min_{\theta} L(\theta). \tag{4.1}$$

The optimiser determines how the parameters are updated using gradient information.

4.2 Adam Optimiser

Adam combines exponentially weighted estimates of the first and second moments of the gradients. Given the gradient $g_t = \nabla_\theta L_t$,

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \tag{4.2}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2. \tag{4.3}$$

After bias correction,

$$\widehat{m}_t = \frac{m_t}{1 - \beta_1^t}, \tag{4.4}$$

$$\widehat{v}_t = \frac{v_t}{1 - \beta_2^t}, \tag{4.5}$$

the parameter update is

$$\theta_{t+1} = \theta_t - \eta \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \epsilon}. \tag{4.6}$$

Adam was studied because its adaptive gradient scaling is widely used for neural-network training. It can be useful when the loss landscape has gradients with different magnitudes across parameter directions.

4.3 L-BFGS Optimiser

L-BFGS is a limited-memory quasi-Newton method. Newton's method uses curvature information through the Hessian matrix:

$$\theta_{k+1} = \theta_k - H_k^{-1} \nabla L(\theta_k). \tag{4.7}$$

Computing and storing the full Hessian is generally expensive for large neural networks. L-BFGS instead constructs an approximation to inverse-Hessian information from a limited history of parameter and gradient differences.

A conceptual L-BFGS update is

$$\theta_{k+1} = \theta_k - \alpha_k B_k \nabla L(\theta_k), \tag{4.8}$$

where B_k approximates inverse-Hessian information and α_k is the step length selected by the line search.

The strong-Wolfe line search tests step lengths using conditions on the objective and its directional derivative. In PyTorch, the L-BFGS closure recomputes the loss and its gradients whenever the optimiser requires an objective evaluation.

4.4 Why Compare Adam and L-BFGS?

The algorithms use different optimisation principles. Adam uses adaptive moment estimates, whereas L-BFGS uses an approximation of curvature information. Comparing them on the same differential equation provides a practical way to examine differences in loss convergence and solution accuracy.

Chapter 5

Dataset and Experimental Configuration

5.1 Training Dataset

The training data consist of collocation points in the unit square and prescribed boundary points.

Table 5.1: Training dataset composition

Point category	**Number of points**
Interior collocation points	8,000
Left boundary points	500
Right boundary points	500
Bottom boundary points	500
Top boundary points	500
Total boundary points	2,000
Total training points	**10,000**

Interior points are sampled in the domain and are used to evaluate the PDE residual. Boundary points carry prescribed Dirichlet values and are used to evaluate the boundary loss.

5.2 Experimental Hyperparameters

Table 5.2: Optimisation configurations used in the reported runs

Parameter	**Adam**	**L-BFGS**
Training points	10,000	10,000
Epochs / outer iterations	10,000	10,000
Learning-rate parameter	0.001	1.0
Boundary loss weight	10	10
Hidden layers	3	3
Neurons per hidden layer	32	32
Activation	Tanh	Tanh
Line search	Not used	Strong Wolfe

Important distinction: the Adam learning rate and the L-BFGS `lr` parameter are not directly equivalent. L-BFGS uses a line-search procedure that can adjust the step length. Also, one L-BFGS outer iteration can involve multiple closure evaluations. Thus, equal epoch and outer-iteration counts do not imply equal computational effort.

5.3 Boundary Sampling Note

The supplied L-BFGS implementation uses evenly spaced boundary coordinates generated using `torch.linspace`. The earlier Adam dataset implementation used random boundary sampling. Unless the Adam run is regenerated with the same boundary coordinates, the comparison is not a strictly controlled optimiser-only experiment. This difference is retained as an experimental limitation.

Chapter 6

Results and Analysis

6.1 L-BFGS: Predicted Solution and Error

Figure 6.1 compares the analytical solution, the L-BFGS prediction and the absolute error distribution.

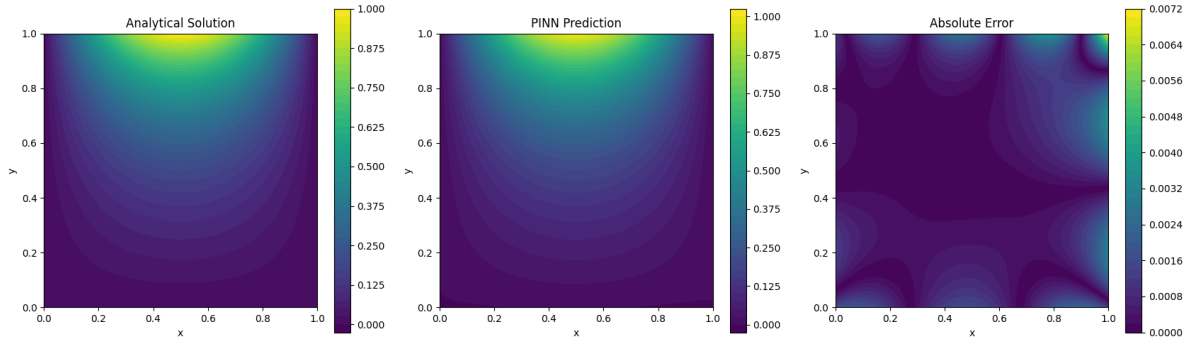


Figure 6.1: Analytical solution, PINN prediction and absolute error for the L-BFGS run with step-size parameter 1.0.

The contour plots show that the predicted solution reproduces the principal spatial structure of the analytical solution. The absolute error is spatially nonuniform, with localised regions of comparatively larger discrepancy.

6.2 Adam: Predicted Solution and Error

Figure 6.2 presents the corresponding analytical solution, PINN prediction and absolute error for the Adam run.

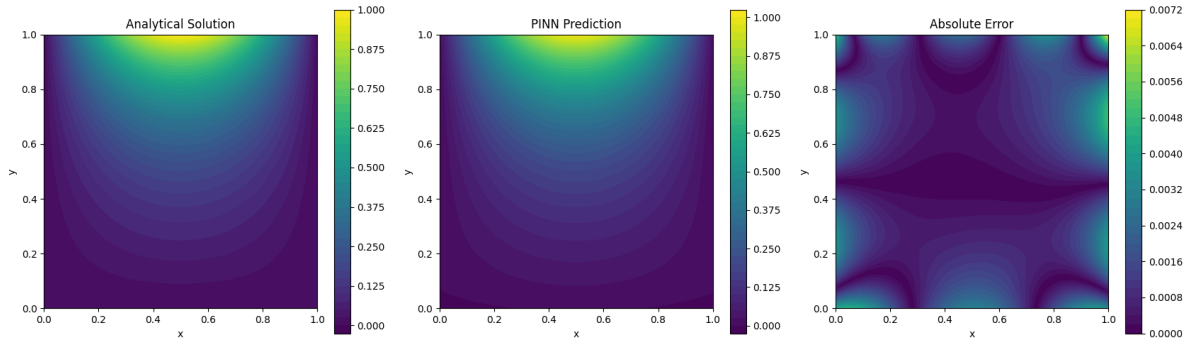


Figure 6.2: Analytical solution, PINN prediction and absolute error for the Adam run with learning rate 0.001.

The prediction visually follows the analytical solution. The error contour provides additional information about where the approximation deviates from the exact solution.

6.3 L-BFGS Training Convergence

Figure 6.3 shows the L-BFGS training history.

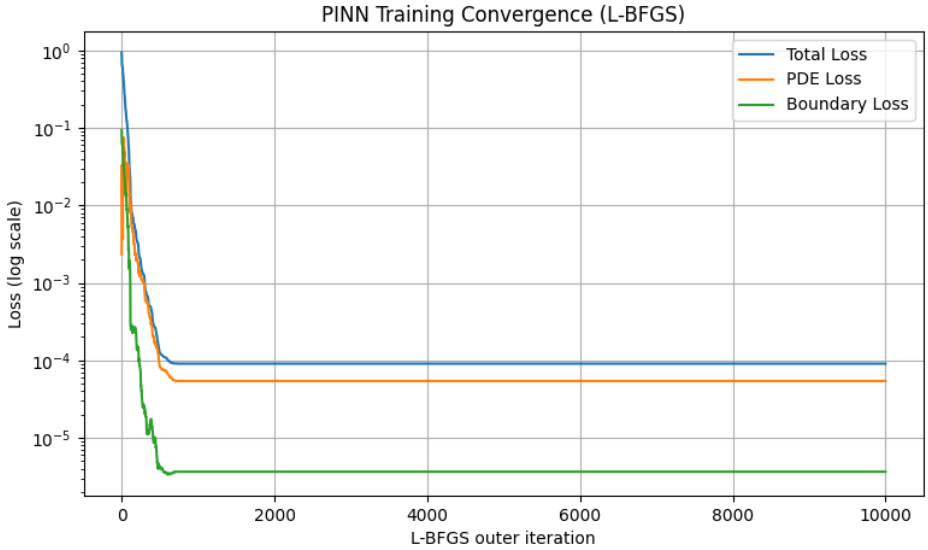


Figure 6.3: L-BFGS convergence history over 10,000 outer iterations.

The curve exhibits a rapid initial decrease in total, PDE and boundary losses, followed by an extended plateau. The earlier L-BFGS experiment with a much smaller step-size parameter, 0.0001, stagnated at a substantially higher loss. Increasing the parameter to 1.0 produced a more effective initial reduction in the objective for this run.

6.4 Adam Training Convergence

Figure 6.4 shows the Adam training history.

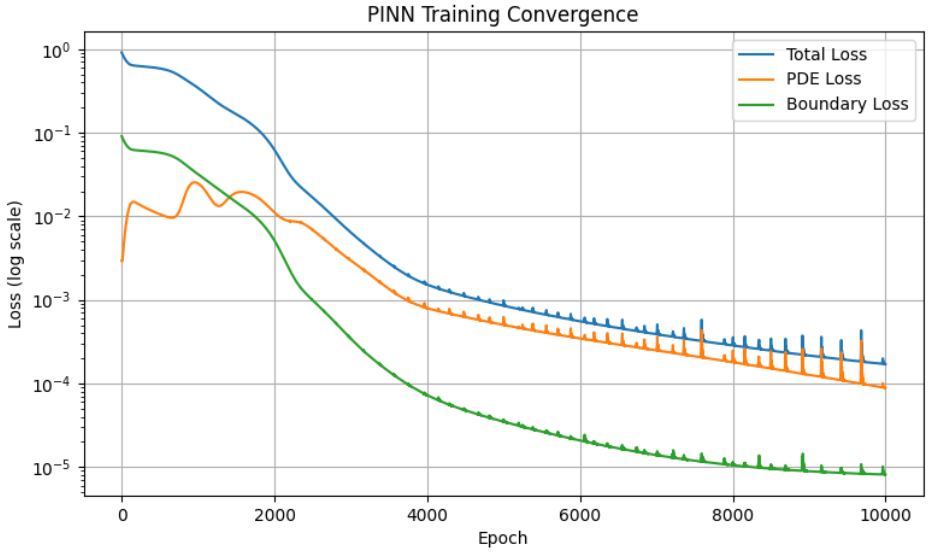


Figure 6.4: Adam convergence history over 10,000 epochs.

The Adam curve shows a gradual downward trend in the total loss, accompanied by recurring local spikes. Both PDE and boundary losses continue to decrease overall. The spikes indicate non-monotonic iteration-to-iteration behaviour; they do not by themselves establish that training has diverged.

6.5 Quantitative Validation

The predicted solutions are evaluated on a 100×100 grid using the analytical solution in Equation 2.8.

The three reported metrics are defined as follows.

Mean Absolute Error

$$\mathrm{MAE} = \frac{1}{N}\sum_{i=1}^{N} |\widehat{u}_i - u_i|. \tag{6.1}$$

Maximum Absolute Error

$$E_{\max} = \max_{1\le i\le N} |\widehat{u}_i - u_i|. \tag{6.2}$$

Relative L^2 Error

$$E_{L^2} = \frac{\|\widehat{\boldsymbol{u}} - \boldsymbol{u}\|_2}{\|\boldsymbol{u}\|_2}. \tag{6.3}$$

The measured validation results are:

Table 6.1: Numerical validation results

Metric	**Adam** (0.001)	**L-BFGS** (1.0)
Mean Absolute Error	0.0010247309	0.00057760574
Maximum Absolute Error	0.0071077373	0.0077121293
Relative L^2 Error	0.0048162495	0.0031214966

6.6 Interpretation of Validation Results

Compared with Adam, L-BFGS reduced the mean absolute error by approximately 43.63% and the relative L^2 error by approximately 35.20% in the reported runs.

However, the maximum absolute error for L-BFGS was approximately 8.51% higher than that for Adam.

This distinction is important. MAE measures average pointwise discrepancy, relative L^2 error measures global error relative to the reference solution, and maximum absolute error measures the largest observed discrepancy. An optimiser can perform better on average while retaining a slightly larger local maximum error.

The results therefore favour L-BFGS for the average and relative global accuracy metrics in these particular runs, while Adam has a small advantage in the maximum pointwise error.

Chapter 7

Discussion

7.1 Observed Optimisation Behaviour

The experiments reveal different convergence patterns.

- **Adam:** gradual loss reduction with recurring spikes. The overall downward trend indicates continued progress, despite non-monotonic behaviour.
- **L-BFGS with** $lr = 0.0001$**:** rapid initial reduction followed by early stagnation at a comparatively high loss.
- **L-BFGS with** $lr = 1.0$**:** substantially greater initial loss reduction, followed by an extended plateau.

These observations show that the optimiser configuration influences the training trajectory. They do not establish that one optimiser is universally superior.

7.2 Limitations

The current comparison has several limitations:

1. The reported results are individual runs, not statistical summaries over multiple random seeds.
2. The boundary-point generation strategies differ between the supplied Adam and L-BFGS implementations.
3. Equal epoch and outer-iteration counts do not imply equal objective evaluations or equal wall-clock time.
4. The comparison uses one PDE, one network architecture and one set of boundary conditions.

5. The training loss and validation error measure different quantities and should not be treated as interchangeable.

7.3 Further Work

The next useful steps are to repeat the experiments with identical training points and initial network parameters, use multiple random seeds, and record wall-clock training time. A systematic dataset-size study could also compare 3,600, 10,000 and 20,000 training points while keeping the remaining experimental conditions fixed.

Chapter 8

Conclusion

A Physics-Informed Neural Network was implemented to approximate the solution of a two-dimensional Laplace boundary-value problem. The governing equation was enforced through a mean-squared PDE residual, while Dirichlet boundary conditions were incorporated through a separate boundary loss.

The project examined two optimisation methods, Adam and L-BFGS, and compared their convergence histories and numerical validation results. Adam displayed gradual convergence with recurring loss oscillations. L-BFGS displayed rapid initial convergence when its step-size parameter was increased to 1.0, followed by an extended plateau.

For the reported runs, L-BFGS achieved lower MAE and relative L^2 error, while Adam achieved a slightly lower maximum absolute error. These findings illustrate why loss curves alone are not sufficient for evaluating a PINN: the resulting solution must also be compared against an independent reference.

The project provides a practical study combining elliptic PDEs, automatic differentiation, neural-network approximation and numerical optimisation. Its main outcome is a reproducible computational framework and an initial comparison of optimisation behaviour for a well-defined differential-equation problem.